Leave Submission API

Solution Design Document — Day-Level Persistence

VERSION 2.3

2026-04-01

Made with GAMMA

Document Control

Author: Bomyung Kim · **Version:** 2.3 · **Date:** 2026-04-01

Version	Date	Description
1.0	2026-03-27	Initial draft — SQL Server production schema
1.2	2026-03-27	SQLite PoC + Streamlit UI added
1.3	2026-03-27	Atomic SubmissionId + concurrency handling
1.4	2026-03-27	API specification section
1.5	2026-03-27	DQ engine + DQResult table
1.6	2026-03-27	Assessment compliance note + dual-mode SubmissionId
1.7	2026-03-29	SQL Server SP layer, UNQ-001 hard reject, Appendix restructure
2.0	2026-03-29	Full restructure to standard SDD template
2.2	2026-04-01	SCD type classification at table level; audit fields CreatedDatetime/UpdatedDatetime added; SCD Type 2 evolution path added

Made with GAMMA

Executive Summary

Purpose

REST API accepting leave submissions (JSON) and persisting to SQL Server at day-by-day granularity. SQLite PoC provided for local dev.

Core Scope

POST/GET endpoints, Mon–Fri decomposition, SQL Server schema, idempotency, atomic transactions, batch persistence, full test coverage.

Bonus Deliverables ⭐

SQLite PoC with Streamlit UI, DQ engine (5 domains, 16 rules), overlap detection, server-generated SubmissionId fallback.

Made with GAMMA

Business Context

Problem Statement

Current approach stores only header-level records, preventing day-level queries for leave balance calculations, overlap detection, and regulatory reporting.

Stakeholders

- **HR Operations** — Accurate leave records and balance visibility
- **Assessment Candidate** — Demonstrate API + DB design skills
- **HRIS** — Source of worker master data
- **Payroll** — Downstream consumer of day-level records

Assumptions

- Working days are Monday–Friday only; public holidays out of scope
- WorkerId is a valid HRIS identifier — no FK enforcement in scope
- Single submission may contain multiple leave types
- submissionId pattern: `LS-YYYY-NNNNNN`; caller may supply or omit
- Quantity per day row is always `1.00`

Constraints

- Python + FastAPI; SQL Server production DB
- Schema fixed by assessment spec
- PoC must run without SQL Server — SQLite used

Made with GAMMA

Functional Requirements

ID	Requirement	Description
FR-01	POST endpoint	Accept leave submission JSON at `POST /api/v1/leave-submissions`
FR-02	Payload validation	Validate required fields, types, and `startDate ≤ endDate`
FR-03	Working-day alignment	Verify `totalWorkingDays` and quantity sum match actual Mon–Fri count
FR-04	Day decomposition	Expand leave period into one record per Mon–Fri working day
FR-05	Atomic persistence	Write LeaveSubmission header + all LeaveDay rows in a single transaction
FR-06	Idempotency	Return HTTP 409 if a duplicate submissionId is submitted
FR-07	GET endpoint	Retrieve submission header and all day records by submissionId
FR-08	Error responses	Return HTTP 400, 409, 422, 500 with descriptive messages
FR-09	SubmissionId generation	Auto-generate `LS-YYYY-NNNNNN` from DB MAX(sequence)+1 if caller omits
FR-10 ⭐	DQ engine	Run 16 data-quality rules across 5 domains on every submission
FR-11 ⭐	Overlap detection	Reject (HTTP 400) submissions that overlap existing leave dates for the same worker
FR-12 ⭐	Streamlit UI	4-page web UI for submit, balance, admin, and DQ dashboard

Made with GAMMA

Non-Functional Requirements

Performance

API response < 500 ms for a 15-day submission under normal load

Scalability

Stateless FastAPI service — horizontally scalable behind a load balancer

Availability

Target 99.9% uptime in production; SQLite PoC for local dev only

Reliability

All DB writes are atomic (transaction + rollback) — no partial data persisted on failure

Auditability

Full day-level traceability: every working day linked back to its parent SubmissionId

Logging

Structured timestamped logs

Testability

Business logic and validation are pure Python with no DB dependency — fully unit-testable; integration tests use in-memory SQLite

Portability

DB layer is swappable (SQLite ↔️ SQL Server) via a single env var — no other code changes required

Security

API runs over HTTP in PoC; TLS 1.2+ required in production; no authentication implemented in scope

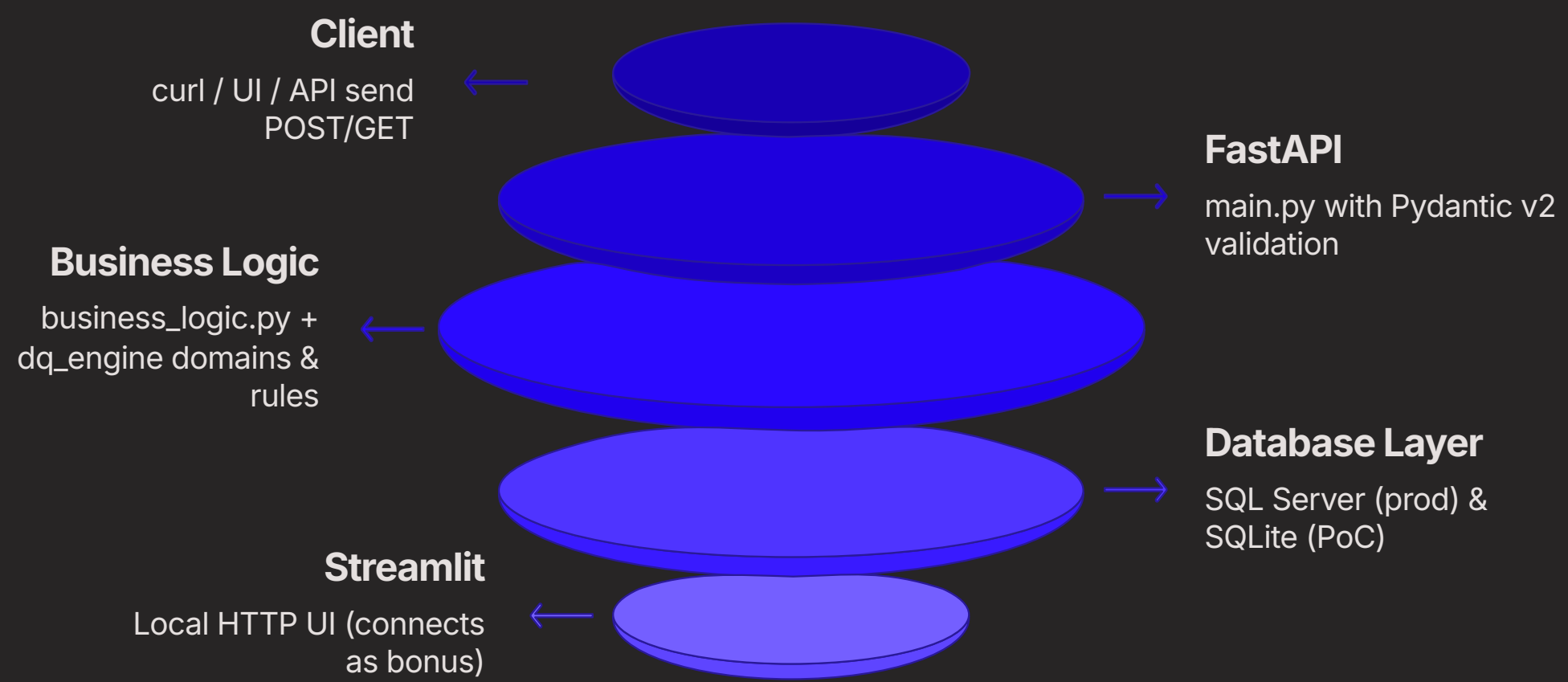
Maintainability

DQ rules are isolated in `dq_engine.py` — adding a rule requires one function addition, no core changes

Made with GAMMA

High-Level Architecture

Style: RESTful API · Monolithic service (PoC) · Swappable DB layer



Client requests flow through FastAPI validation and business logic layers before reaching either SQL Server (production) or SQLite (PoC). Streamlit UI connects locally via HTTP as a bonus deliverable.

Technology Stack

- **API:** FastAPI (Python) + Pydantic v2
- **Business Logic:** Pure Python (`business_logic.py`, `dq_engine.py`)
- **DB — Production:** SQL Server via pyodbc + `usp_PersistLeaveSubmission` SP
- **DB — PoC:** SQLite via sqlite3 (stdlib)
- **UI — PoC ⭐:** Streamlit
- **Testing:** Pytest + FastAPI TestClient (in-memory SQLite)
- **Runtime:** Uvicorn (ASGI)

Key Design Points

- DB layer swappable via single env var — no code changes
- DQ rules isolated in `dq_engine.py` — one function per new rule
- Business logic fully unit-testable without DB dependency
- Stateless service — horizontally scalable behind load balancer

Made with GAMMA

API Specification

Endpoints

Method	Path	Description
POST	`/api/v1/leave-submissions`	Submit a leave request
GET	`/api/v1/leave-submissions/{id}`	Retrieve submission + day records
GET	`/health`	Liveness check

POST — Status Codes

Code	Scenario	Description
201	Success	Submission created; all day rows written
400	Business rule violation	Working-day mismatch, quantity error, or UNQ-001 DQ rejection
409	Duplicate	Caller-supplied submissionId already exists
422	Validation failure	Missing fields, wrong types, or `startDate > endDate`
500	Server error	DB failure — transaction rolled back, no partial data written

GET — Status Codes

Code	Scenario	Description
200	Submission found	Returns header + day records
404	Not found	submissionId not found

`submissionId` is optional — if omitted, server generates `LS-YYYY-NNNNNN` from `MAX(sequence)+1`.

POST — Request Body


```
{
 "leaveSubmission": {
 "submissionId": "LS-2026-000123",
 "submittedDate": "2026-02-15",
 "status": "Submitted",
 "worker": {
 "workerId": "W123456",
 "employeeNumber": "90030366",
 "sourceSystem": "HRIS"
 },
 "leavePeriod": {
 "startDate": "2026-03-02 00:00:00.00",
 "endDate": "2026-03-20 23:59:59.99",
 "totalWeeks": 3,
 "totalWorkingDays": 15
 },
 "leaveDetails": [
 {
 "leaveTypeCode": "AL",
 "leaveTypeDescription": "Annual Leave",
 "leaveCategory": "Paid",
 "unitOfMeasure": "Days",
 "quantity": 15
 }
 ],
 "approver": {
 "approverId": "M987654",
 "approvalStatus": "Pending"
 },
 "comments": "Planned annual leave for personal travel."
 }
}
```


Made with GAMMA

Business Logic & Data Model

Processing Flow

1. **Parse & Validate** — Pydantic v2 validates fields; HTTP 422 on failure
2. **Date Order Check** — `startDate ≤ endDate`; HTTP 422 if violated
3. **SubmissionId Resolution** — Duplicate check → HTTP 409, or server generates `LS-YYYY-NNNNNN`
4. **Alignment Check** — Mon–Fri count must equal `totalWorkingDays` and quantity sum; HTTP 400 on mismatch
5. **DQ Checks** ⭐ — 16 rules across 5 domains; UNQ-001 overlap → HTTP 400, nothing persisted
6. **Day Decomposition** — One `LeaveDayRecord` per Mon–Fri at `Quantity = 1.00`
7. **Atomic DB Write** — Header + day rows in single transaction; rolled back on error
8. **201 Response** — Returns `submissionId`, `workerId`, `totalWorkingDaysCreated`, `leaveDays`, `dq_issues`

Working-Day Decomposition

2 Mar – 20 Mar 2026 → 15 days (Mon–Fri only)

Date	Status
03/02 – 03/06 (Mon–Fri)	✓ Persisted
03/07 – 03/08 (Sat–Sun)	— Skipped
03/09 – 03/13 (Mon–Fri)	✓ Persisted
03/14 – 03/15 (Sat–Sun)	— Skipped
03/16 – 03/20 (Mon–Fri)	✓ Persisted

Core Tables

- **LeaveSubmission** — Header (SCD Type 1); audit fields `CreatedDatetime` / `UpdatedDatetime`
- **LeaveDay** — One row per working day (SCD Type 1); audit fields `CreatedDatetime` / `UpdatedDatetime`
- **DQResult** — Stores all DQ check outcomes per submission

Made with GAMMA

DQ Engine & Appendices

⭐ **Bonus deliverable** — 16 rules across 5 domains, executed on every submission. UNQ-001 (overlap) → HTTP 400 hard reject; all others are soft warnings.

Uniqueness

UNQ-001: Overlapping leave dates for same worker

Completeness

CMP-001 to CMP-003: Required fields present

Validity

VAL-001 to VAL-005: Date formats, code values, workerId pattern

Consistency

CNS-001 to CNS-003: Status vs approver, quantity vs working days

Accuracy

ACC-001 to ACC-004: Date arithmetic, sequence number format

Appendices

- **A** — SQL Server DDL (`LeaveSubmission`, `LeaveDay`, `DQResult`)
- **B** — `usp_PersistLeaveSubmission` stored procedure
- **C** — Sample payloads (success, duplicate, working-day mismatch, overlap)
- **D** — Running the PoC (SQLite + Streamlit)
- **E** — SCD Type 2 evolution path: add `ValidFrom`, `ValidTo`, `IsCurrent`, unique index on (`SubmissionId`, `ValidFrom`), update SP to close prior version on change

Made with GAMMA